

RAGScope

Deployment Blocker Report

Python 3.14 / asyncio Incompatibility on Render Free Tier

Prepared by Imtiaz | May 17, 2026 | Confidential

Executive Summary

RAGScope is a fully built, publicly accessible RAG benchmarking tool. The application code is correct, the evaluation pipeline is proven, and 106 automated tests pass. A single environmental incompatibility prevents production deployment: Render's free tier runs Python 3.14, which breaks the asyncio internals that RAGAS (our evaluation framework) depends on. This report documents the problem, the diagnostic journey, the evidence, and three concrete resolution paths.

01

What RAGScope Is

RAGScope is a deterministic evaluation pipeline for Retrieval-Augmented Generation. It is not a RAG application - it is a measurement instrument. Users paste any document corpus, choose from four retrieval strategies, and receive ground-truth data on which strategy performs best for their specific content.

The value proposition is simple: most teams pick a RAG strategy based on intuition or vendor recommendations. RAGScope replaces that guesswork with empirical data - faithfulness scores, context utilization ratios, answer relevancy metrics, and per-strategy latency - all computed against their own corpus, with their own questions.

Who benefits

- AI engineers evaluating RAG pipelines before production commits
- Researchers comparing retrieval architectures on domain-specific corpora
- Product teams validating that LLM answers stay grounded in their documentation
- Anyone who wants to know: for my data, which RAG strategy wins?

The four retrieval strategies

Strategy	Mechanism	Best for
Naive RAG	Direct cosine similarity on query embedding	Baseline, short corpora
HyDE	LLM generates hypothetical answer first, then embeds it	Abstract or conceptual questions
Multi-query	LLM generates 3-5 query variants, merges results	Ambiguous or multi-faceted queries
Hybrid BM25 + Dense	Sparse + dense search fused with Reciprocal Rank Fusion	Keyword-heavy technical docs

Contextual compression is available as a post-retrieval processor on top of any strategy - not a fifth strategy itself.

What We Built

RAGScope took approximately three intensive development sessions to build from scaffold to deployed product. Every component was built with the constraint that it must be maintainable, testable, and extensible.

Stack

Layer	Technology	Role
Backend API	FastAPI + Python 3.11	Request handling, background task dispatch
Vector store	pgvector via Supabase	Embedding storage and cosine similarity search
Sparse index	rank-bm25 (pure Python)	Keyword retrieval for hybrid strategy
Embeddings	text-embedding-3-small	1536-dimension query and chunk embeddings
LLM judge	gpt-4o-mini	Retrieval, generation, and RAGAS evaluation
Eval framework	RAGAS 0.1.21 (pinned)	Faithfulness, context utilization, answer relevancy
Frontend	Next.js 14 + Tailwind	4-step benchmark flow, recharts visualizations
Tracing	LangSmith	Observability across retrieval and eval calls
Deployment	Render (backend) + Vercel (frontend)	Free-tier hosting

What is complete and verified

- 106 backend tests passing (pytest), covering retrieval, eval, ingest, and chat
- 4 retrieval strategies with param schemas auto-discovered via registry pattern
- RAGAS evaluation running as async background task (non-blocking)
- Three-tier access system: Dev (unlimited), Guest (12 runs/day), BYOK (user pays)
- Full frontend: multi-strategy checkbox selection, parallel result streaming per strategy
- Live chat endpoint (POST /chat) with backend-enforced daily limits
- pgvector + Supabase integration with asyncpg, transaction pooler, SSL
- Contextual compression as orthogonal post-retrieval toggle
- Render + Vercel deployment, LangSmith tracing, CORS locked to production domain

- Session H benchmark: 40 runs across 10 questions x 4 strategies - HyDE wins at 0.9073

03

Where We Are Stuck

The application is deployed. The frontend at ragscope.vercel.app is live. The backend at ragscope-backend.onrender.com responds to all API calls. Ingestion works. Retrieval works. Answer generation works. The single broken component is RAGAS metric scoring in the background evaluation task.

The core problem

RAGAS 0.1.21 uses `asyncio.timeout()` internally during metric computation. Python 3.14 made `asyncio.timeout()` require an active Task context. Render's free tier runs Python 3.14. When RAGAS metric scoring runs inside our background task thread on Render, the Task context is not preserved consistently, causing all three metric computations to raise `RuntimeError` and return `NaN`. The run completes, but all quality metrics are null.

The practical consequence: users can run benchmarks on Render, but the faithfulness, context utilization, and answer relevancy scores that are the entire point of the tool all come back empty. The pipeline executes correctly but produces no measurement value.

04

How We Proved It

We did not assume the root cause. We ran a systematic diagnostic campaign across 8 development sessions, each narrowing the hypothesis. The table below summarises every fix attempted, what it changed, and the verdict.

Session	Hypothesis	Fix attempted	Verdict
C (first)	Background task never fires	Confirmed <code>loop.create_task()</code> wrapper	PARTIAL - task fires but metrics fail
C (fix 1)	RAGAS metrics bundled call fails	Per-metric isolation with individual <code>evaluate()</code> calls	PARTIAL - faithfulness passes, cu/ar fail
C (fix 2)	Post-commit cleanup noise	<code>completed_committed</code> guard added	Fixed noise, core issue remains
Python pin	Python 3.14 incompatibility	<code>runtime.txt</code> with <code>python-3.11.9</code>	FAILED - Render free tier ignores <code>runtime.txt</code>
Env var	Python version via env	<code>PYTHON_VERSION=3.11.9</code> env var	FAILED - buildpack ignores env var too

timeout=None	asyncpg timeout triggers asyncio.timeout()	Pass timeout=None to asyncpg.create_pool()	FAILED - asyncpg calls compat.timeout() unconditionally
Direct conn	Pool initialization fails, direct conn avoids it	Replace make_task_pool() with asyncpg.connect()	FAILED - asyncpg.connect() has same internal path
psycpg 2	Replace asyncpg with sync driver in background task	psycpg2-binary for all DB calls in eval path	PARTIAL - single runs work, parallel load fails intermittently
Local run	Validate on Python 3.11	Run Session H against local backend	CONFIRMED WORKING - all 40 runs, real metrics

Table 1: Chronological diagnostic attempts with outcomes

The smoking gun

The decisive evidence came from two observations taken together:

- On the local Python 3.11 backend: all 40 Session H runs completed with real metrics. HyDE scored 0.9073, Naive 0.9046. The pipeline is provably correct.
- On Render Python 3.14: the Render log showed the exact traceback - asyncio/timeouts.py line 88, `__aenter__`, raise `RuntimeError('Timeout should be used inside a task')` - originating from inside RAGAS's `AnswerRelevancy._ascore` coroutine.

```
Exception raised in Job[0]: RuntimeError(Timeout should be used inside a task)
/opt/render/.../ragas/executor.py:64: RuntimeWarning: coroutine 'AnswerRelevancy._ascore'
was never awaited
```

RAGAS 0.1.x was written for Python 3.10-3.11. The `asyncio.timeout()` context manager it relies on internally was added in Python 3.11 but made stricter in 3.12+ and further restricted in 3.14 to require an active Task. This is a library-runtime mismatch that cannot be patched from application code.

05

Benchmark Results (Local, Confirmed)

To establish a baseline and validate the full pipeline, Session H ran 40 benchmarks locally against a motor vehicle corpus (10 questions x 4 strategies). All metrics are real RAGAS scores on Python 3.11.

Strategy	Faithfulness	Context util.	Answer rel.	Latency (mean)	Weighted score
HyDE	1.000 (n=10)	1.000 (n=9)	0.989 (n=8)	23,674 ms	0.9073
Naive RAG	1.000 (n=10)	1.000 (n=8)	0.980 (n=9)	23,486 ms	0.9046
Hybrid	0.990 (n=10)	0.812 (n=8)	0.982 (n=8)	22,913 ms	0.8266

Multi-query	1.000 (n=7)	1.000 (n=9)	0.974 (n=9)	25,489 ms	0.8129
-------------	-------------	-------------	-------------	-----------	--------

Table 2: Session H results - 40 runs on local Python 3.11. Weighted score: faithfulness 40%, context utilization 30%, answer relevancy 30%. Null metrics scored as 0.0 in weighted calculation.

Key finding: HyDE wins, but barely

HyDE's 0.0027 weighted score advantage over Naive is within RAGAS metric noise. The meaningful finding is the gap below: Hybrid drops to 0.8266 due to poor context utilization on 3 questions (BM25 surfaces partially-relevant chunks for this short corpus), and Multi-query reaches 0.8129 due to 3 null faithfulness scores (RAGAS struggled to evaluate its multi-chunk outputs).

06

Proposed Solutions

Three paths exist to resolve the Render Python 3.14 incompatibility. They are listed in order of preference.

Option A - Upgrade to Render Starter tier

Cost: \$7/month

- Render's paid tiers support manual Python version selection via the Settings UI.
- Select Python 3.11.9, trigger a redeploy - everything works immediately.
- No code changes required. The application is already correct.
- Also eliminates the free-tier spin-down delay (50s+ cold starts) that hurts UX.
- Estimated time to implement: 10 minutes.

Verdict: Recommended. Cleanest resolution with no technical risk.

Option B - Migrate to a Python 3.11 host

Cost: Free alternatives exist

- Railway.app, Fly.io, and Google Cloud Run all support explicit Python version pinning at no cost for low-traffic services.
- The render.yaml already exists in the repo and can be adapted to any of these platforms.
- Estimated time to implement: 1-2 hours for platform migration.

Verdict: Good alternative if staying on a free tier is a hard constraint.

Option C - Pin RAGAS to a Python 3.14 compatible version

Cost: Code change required

- RAGAS 0.2.x rewrote the async evaluation engine. It may no longer rely on `asyncio.timeout()` in the same way.
- Risks: RAGAS 0.1.x and 0.2.x have incompatible APIs. `context_utilization` is a 0.1.x reference-free metric not available in 0.2.x under the same name.
- Would require updating `ragas_runner.py`, re-validating all metric names, and re-running the test suite.
- Estimated time to implement: 4-8 hours with testing.

Verdict: Viable but carries API migration risk and testing overhead.

07

Current State and Next Steps

The table below gives an honest snapshot of every system component as of May 17, 2026.

Component	Status	Notes
Backend API (FastAPI)	Live on Render	All endpoints respond correctly
Frontend (Next.js)	Live on Vercel	ragscope.vercel.app - full 4-step flow
Ingest pipeline	Working	PDF + TXT, 10MB limit, corpus hashing
Retrieval (all 4 strategies)	Working	Locally and on Render
Answer generation	Working	gpt-4o-mini, locally and on Render
RAGAS evaluation (local)	Working	Python 3.11, all 3 metrics confirmed
RAGAS evaluation (Render)	BROKEN	Python 3.14 asyncio incompatibility
Chat endpoint (POST /chat)	Live	5/day guest limit, backend-enforced
Rate limiting	Live	12 runs/day guest, fingerprint + IP
Dev tier bypass	Live	SHA-256 token, unlimited access
LangSmith tracing	Live	Full trace per benchmark run
106 backend tests	Passing	pytest, all green
README + render.yaml	Committed	Deployment docs complete

Recommended next steps in order

1. Upgrade Render to Starter tier (\$7/month) - resolves the blocker in 10 minutes

2. Trigger a redeploy and run the parallel probe (4 strategies, 1 question) to confirm RAGAS metrics return non-null
3. Re-run Session H on Render (the 40-question sweep) to produce the canonical production benchmark results
4. Remove the debug print probes from benchmark.py once production is confirmed stable
5. Address BYOK unlimited chat (currently still limited by backend chat_count) as a follow-on session

RAGScope | github.com/imtiazx/ragscope | ragscope.vercel.app | May 2026